

CHUYÊN ĐỀ . DUYỆT TOÀN BỘ NÂNG CAO

1. Phương pháp (thuật toán quay lui Backtracking)

Để **giải bài toán bằng thuật toán quay lui** cần thực hiện các công việc quan trọng sau:

- * Tìm cách biểu diễn nghiệm của bài toán dưới dạng một dãy các đối tượng được chọn dần từng bước ($x_1, x_2, \dots, x_n$).

- * Xác định được tập S_i các khả năng được chọn làm thành phần thứ i của nghiệm. Chọn cách thích hợp để biểu diễn S_i .

- * Tìm các điều kiện để các nghiệm đã chọn là các nghiệm của bài toán.

Một số lưu ý khi xây dựng thuật toán quay lui:

- * Phải duyệt qua mọi phương án của bài toán có thể chứa nghiệm (vét cạn).

- * Tránh trường hợp duyệt trùng lặp các khả năng đã duyệt.

Để giải các bài toán bằng thuật toán quay lui, thông thường ta thường dùng thủ tục đệ quy Try(i : Integer) để chọn thành phần nghiệm x_i .

Có ba dạng cơ bản trong các thuật toán quay lui:

Dạng 1: Tìm tất cả các nghiệm.

Dạng 2: Tìm một nghiệm.

Dạng 3: Tìm nghiệm tối ưu thỏa mãn điều kiện.

Ta lần lượt xét cách giải các dạng trên bằng phương pháp quay lui:

2. Một số bài toán sử dụng phương pháp quay lui

- Liệt kê dãy nhị phân độ dài n .
- Xác định tổ hợp chập k của tập n phần tử.
- Liệt kê các chỉnh hợp không lặp chập k .
- Bài toán phân tích số, (phân tích một số bằng tổng của các số).
- Bài toán xếp N quân hậu trên một ván cờ.
- Bài toán người du lịch.
- Bài toán đổi tiền (đổi các đồng tiền có tổng bằng S).
- Bài toán xếp va li.
- Bài toán con mã đi tuần...

PHƯƠNG PHÁP QUAY LUI (BACKTRACKING)

1. Dạng 1: Tìm tất cả các nghiệm

```
void Try(i: Integer)
{
    //Thủ tục này thử cho  $x_i$  nhận lần lượt các giá trị mà nó có thể nhận
    int j;
    for (<mọi giá trị j có thể gán cho  $x_i$ > )
    {
        <Thử cho  $x_i := j$ >;
        If (< $x_i$  là phần tử cuối cùng trong cấu hình>)
            <Thông báo cấu hình tìm được>
        Else
        {
            <Ghi nhận việc cho  $x_i$  nhận giá trị j (Nếu cần)>;
            Try(i + 1); //Gọi đệ quy để chọn tiếp  $x_{i+1}$ 
            <Nếu cần, bỏ ghi nhận việc thử  $x_i := j$ , để thử giá trị khác>;
        }
    }
}
End;
```

Thuật toán quay lui thường bắt đầu bằng lời gọi Try(1)

2. Dạng 2 : Tìm một nghiệm

```
void Try(i: Integer);
//Thủ tục này thử cho  $x_i$  nhận lần lượt các giá trị mà nó có thể nhận
{
    for (<mọi giá trị j có thể gán cho  $x_i$ > )
    {
        <Thử cho  $x_i := j$ >;
        If (< $x_i$  là phần tử cuối cùng trong cấu hình>)
        {
            <Thông báo cấu hình tìm được>
            exit(0); //thoát khỏi chương trình
        }
        Else {
            <Ghi nhận việc cho  $x_i$  nhận giá trị j (Nếu cần)>;
            Try(i + 1); //Gọi đệ quy để chọn tiếp  $x_{i+1}$ 
            <Nếu cần, bỏ ghi nhận việc thử  $x_i := j$ , để thử giá trị khác>;
        }
    }
}
```

2. Dạng 3. Tìm nghiệm tối ưu thoả mãn điều kiện

```
void khoitao;  
{  
    <Khởi tạo một cấu hình bất kỳ CAUHINH>;  
}
```

//Thủ tục này thử chọn cho x_i tất cả các giá trị nó có thể nhận

```
void Try(i: Integer);  
Begin  
    for (Mọi giá trị  $j$  có thể gán cho  $x_i$ )  
    {  
        <Thử cho  $x_i := j$ >;  
        If (Việc thử trên vẫn còn hi vọng tìm ra cấu hình tốt hơn CAUHINH)  
        { If ( $x_i$  là phần tử cuối cùng trong cấu hình)  
            <cập nhật CAUHINH>  
        Else  
        {  
            <Ghi nhận việc thử  $x_i = j$  (nếu cần)>;  
            Try(i + 1);      //Gọi đệ quy, chọn tiếp  $x_{i+1}$   
            <Bỏ ghi nhận việc thử cho  $x_i = j$  (nếu cần)>;  
        }  
    }  
}
```

```
// trong chương trình chính  
{    khoitao; Try(1);  
    <Thông báo cấu hình tối ưu CAUHINH>;  
}
```

Kỹ thuật nhánh cận thêm vào cho thuật toán quay lui khả năng đánh giá theo từng bước, nếu tại bước thứ i , giá trị thử gán cho x_i không có hi vọng tìm thấy cấu hình tốt hơn cấu hình CAUHINH thì thử giá trị khác ngay mà không cần phải gọi đệ quy tìm tiếp hay ghi nhận kết quả, vì nếu có tiếp tục thì việc làm đó là vô nghĩa và làm cho chi phí của bài toán càng lớn. Nghiệm của bài toán sẽ được làm tốt dần, bởi khi tìm ra một cấu hình mới (tốt hơn CAUHINH), ta không in kết quả ngay mà sẽ cập nhật CAUHINH bằng cấu hình mới vừa tìm được.

BÀI TẬP :

Bài 1: Xây dựng các chỉnh hợp chập K của N phần tử 1,2,3,...,N ($0 < K < N$)

Dữ liệu vào: file CHINHHOP.INP gồm 2 số N, K

Dữ liệu ra : file CHINHHOP.OUT gồm nhiều dòng, mỗi dòng là 1 tổ hợp các phần tử trong tổ hợp cách nhau ít nhất 1 dấu cách

chinhhop.inp	chinhhop.out
3 2	1 2 1 3 2 1 2 3 3 2

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int n,k, x[100],D[100];
```

```
void incauhinh()
```

```
{
```

```
    for(int l=1;l<=k;l++)
```

```
        cout<<x[l]<<" ";
```

```
    cout<<endl;
```

```
}
```

```
void Try(int i)
```

```
{
```

```
    int j;
```

```
    for(j=1;j<=n;j++)
```

```
        if(D[j])
```

```
        {
```

```
            x[i]=j;
```

```
            D[j]=0;
```

```
            if(i==k) incauhinh();
```

```
            else
```

```
                if(i<k) Try(i+1);
```

```
            D[j]=1;
```

```
        }
```

```
}
```

```
int main()
```

```
{
```

```
    freopen("CHINHHOP.INP","r",stdin);
```

```
    freopen("CHINHHOP.OUT","w",stdout);
```

```
    memset(D,1,sizeof(D));
```

```
    cin>>n>>k;
```

```
    Try(1);
```

```
    return 0;
```

```
}
```

- Biến mảng x[100] dùng để lưu cấu hình (kết quả)
- Biến mảng D[100] dùng để đánh dấu các phần tử đã chọn (1 là chưa chọn, 0 đã chọn)
(Dùng biến đánh dấu D khi các phần tử chọn không lặp lại)

Nếu phần tử j chưa được chọn

Chọn phần tử j

Nếu chọn chưa đủ k phần tử thì chọn phần tử tiếp theo

Trả lại trạng thái phần tử j chưa được chọn

Khởi tạo mảng D ban đầu tất cả các phần tử chưa được chọn là 1.

Tìm phần tử đầu tiên cho cấu hình x

Bài 2 : Xây dựng các chỉnh hợp lắp chập K của N phần tử 1,2,3,...,N ($0 < K < N$) (cũn gọi là bộ mẫu N phần tử)

Dữ liệu vào: file CHLAP.INP gồm 2 số N, K

Dữ liệu ra : file CHLAP.OUT gồm nhiều dòng, mỗi dòng là 1 tổ hợp các phần tử trong tổ hợp cách nhau ít nhất 1 dấu cách

CHLAP.INP	CHLAP.OUT
3 2	1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3

Gợi ý:

- Các phần tử có thể được chọn lặp lại nên không cần mảng đánh dấu D

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int n,k, x[100];
```

```
//D[100];
```

```
void incauhinh()
```

```
{
```

```
    for(int l=1;l<=k;l++)
```

```
        cout<<x[l]<<" ";
```

```
    cout<<endl;
```

```
}
```

```
void Try(int i)
```

```
{
```

```
    int j;
```

```
    for(j=1;j<=n;j++)
```

```
        // if(D[j])
```

```
        {
```

```
            x[i]=j;
```

```
            // D[j]=0;
```

```
            if(i==k) incauhinh();
```

```
            else
```

```
                if(i<k) Try(i+1);
```

```
            // D[j]=1;
```

```
        }
```

```
}
```

```
int main()
```

```

{
    freopen("CHLAP.INP","r",stdin);
    freopen("CHLAP.OUT","w",stdout);
    //memset(D,1,sizeof(D));
    cin>>n>>k;
    Try(1);
    return 0;
}

```

Bài 3 : Xây dựng các tổ hợp chập K của N phần tử 1,2,3,...,N ($0 < K < N$)

Dữ liệu vào: file TOHOP.INP gồm 2 số N, K

Dữ liệu ra : file TOHOP.OUT gồm nhiều dòng, mỗi dòng là 1 tổ hợp các phần tử trong tổ hợp cách nhau ít nhất 1 dấu cách

Tohop.inp	Tohop.out
4 3	1 2 3 1 2 4 1 3 4 2 3 4

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int n,k, x[100];
```

```
//D[100];
```

```
void incauhinh()
```

```
{
    for(int l=1;l<=k;l++)
        cout<<x[l]<<" ";
    cout<<endl;
}
```

```
void Try(int i)
```

```
{
    int j;
    for(j=x[i-1]+1;j<=n-k+i;j++)
        // if(D[j])
        {
            x[i]=j;
            // D[j]=0;
            if(i==k) incauhinh();
            else
                if(i<k) Try(i+1);
            // D[j]=1;
        }
}
```

```
int main()
{
```

Do phần tử sau trong x lớn hơn phần tử phía trước nên phần tử tại vị trí i nhận giá trị từ $x[i-1] + 1$ đến tối đa là $n - k + i$.

```

freopen("CHINHHOP.INP","r",stdin);
freopen("CHINHHOP.OUT","w",stdout);
//memset(D,1,sizeof(D));
cin>>n>>k;
x[0]=0;
Try(1);
return 0;
}

```

BÀI TẬP

Bài toán. BÀI TOÁN PHÂN TÍCH SỐ 1

Hãy tìm tất cả các cách phân tích số n thành tổng của các số nguyên dương, các cách phân tích là hoán vị của nhau chỉ tính là 1 cách.

Input: file văn bản PHANTICH.INP chứa số nguyên dương n ($n \leq 30$).

Output: file văn bản PHANTICH.OUT ghi các cách phân tích số n .

PHANTICH.INP	PHANTICH.OUT
5	$5 = 1+1+1+1+1$ $5 = 1+1+1+2$ $5 = 1+1+3$ $5 = 1+2+2$ $5 = 1+4$ $5 = 2+3$ $5 = 5$

Bài toán. BÀI TOÁN PHÂN TÍCH SỐ 2

Hãy tìm tất cả các cách phân tích số n thành tổng của các số nguyên dương, các số dương chỉ được lấy 1 lần, các cách phân tích là hoán vị của nhau chỉ tính là 1 cách.

Input: file văn bản PHANTICH.INP chứa số nguyên dương n ($n \leq 30$).

Output: file văn bản PHANTICH.OUT ghi các cách phân tích số n .

PHANTICH.INP	PHANTICH.OUT
5	$5 = 1+4$ $5 = 2+3$ $5 = 5$

Bài toán. BÀI TOÁN PHÂN TÍCH SỐ 3

Hãy tìm cách phân tích số n thành tổng của các số nguyên dương, sao cho tích các số nguyên dương này là lớn nhất.

Input: file văn bản PHANTICH.INP chứa số nguyên dương n ($n \leq 30$).

Output: file văn bản PHANTICH.OUT ghi các cách phân tích số n .

PHANTICH.INP	PHANTICH.OUT
5	5 = 2+3

Bài toán. Trò chơi trên băng giấy

Trên một băng giấy người ta kẻ N ô vuông liên tiếp nhau theo chiều ngang. Các ô vuông được đánh số từ 1 đến N theo chiều từ trái sang phải ($1 < N < 50$). Tại ô vuông số 1 có đặt một quân cờ. Một nước đi trong trò chơi là dịch chuyển quân cờ về phía bên phải 1 hoặc 2 ô vuông.

Hãy đếm và liệt kê tất cả các cách dịch chuyển quân cờ từ ô vuông số 1 tới ô vuông N (Ký hiệu: ký hiệu 'O' mô tả dịch chuyển quân cờ về phía bên phải 1 ô vuông, ký hiệu 'T' mô tả dịch chuyển quân cờ về phía bên phải 2 ô vuông).

Dữ liệu vào là tệp văn bản BANG.INP ghi số nguyên dương N . Dữ liệu ra là tệp văn bản BANG.OUT có cấu trúc:

- Dòng thứ nhất ghi số nguyên M là số cách dịch chuyển quân cờ từ ô vuông 1 đến ô vuông N .
- M dòng tiếp theo mỗi dòng ghi một cách dịch chuyển trong số M cách di chuyển quân cờ từ ô số 1 tới ô số N .

*			
1	2	3	4

BANG.INP	BANG.OUT
4	3 OOO OT TO

Bài toán. BÀI TOÁN MÁY RÚT TIỀN TỰ ĐỘNG ATM 1

Một máy ATM hiện có n ($n \leq 20$) tờ tiền có mệnh giá $t_1, t_2, t_3, \dots, t_n$. Hãy đưa ra cách trả tiền đúng bằng s .

Input: Tệp ATM.INP có dạng:

- Dòng đầu là hai số n và s .
- Dòng thứ hai gồm n số $t_1, t_2, t_3, \dots, t_n$

Output: Tệp ATM.OUT có dạng: Nếu có thể trả đúng s thì đưa ra cách trả, nếu không ghi -1.

ATM.INP	ATM.OUT
11 490 20 200 10 20 20 50 50 50 50 100 100	200 20 20 50 100 100

Bài toán. Bài toán cái túi 1 (Xếp va Li, xếp Ba lô)

Có N vật (mặt hàng), với mỗi vật ta biết trọng lượng và giá trị của nó. Hãy xác định các vật cần để xếp vào một ba lô có sức chứa tối đa là M sao cho giá trị chứa trong ba lô là lớn nhất.

Dữ liệu vào: Tập văn bản **balo.inp**:

- Dòng đầu tiên: hai giá trị nguyên dương N và M .
- N dòng tiếp theo, mỗi dòng chứa hai giá trị nguyên dương d và v cho mỗi vật, trong đó d là trọng lượng, v là giá trị tính theo một đơn vị trọng lượng của vật đó (đơn giá). Các số cách nhau qua dấu cách.

Dữ liệu ra: Tập văn bản tên **balo.out**: ghi một số là tổng giá trị lớn nhất thu được.

Balo.inp	Balo.out
5 6 2 3 1 1 3 5 1 2 3 2	10

Bài toán. Bài toán người du lịch

Cho n thành phố đánh số từ 1 đến n và m tuyến đường giao thông hai chiều giữa chúng, mạng lưới giao thông này được cho bởi bảng C cấp $n \times n$, ở đây $C_{ij} = C_{ji}$ = Chi phí đi đoạn đường trực tiếp từ thành phố i đến thành phố j . Giả thiết rằng $C_{ii} = 0$ với mọi i , $C_{ij} = +\infty$ nếu không có đường trực tiếp từ thành phố i đến thành phố j .

Một người du lịch xuất phát từ thành phố 1, muốn đi thăm tất cả các thành phố còn lại mỗi thành phố đúng 1 lần và cuối cùng quay lại thành phố 1. Hãy chỉ ra cho người đó hành trình với chi phí ít nhất.

Input: file văn bản TOURISM.INP

- Dòng 1: Chứa số thành phố n ($1 \leq n \leq 20$) và số tuyến đường m trong mạng lưới giao thông.
- m dòng tiếp theo, mỗi dòng ghi số hiệu hai thành phố có đường đi trực tiếp và chi phí đi trên quãng đường đó (chi phí này là số nguyên dương ≤ 100).

Output: file văn bản TOURISM.OUT

- Dòng đầu tiên ghi hành trình.
- Dòng thứ 2 ghi chi phí tìm được.

Ví dụ:

TOURISM.INP	TOURISM.OUT
4 6 1 2 3 1 3 2 1 4 1	1->3->2->4->1 6

2 3 1	
2 4 2	
3 4 4	

Bài 7: Đường đi

Sơ đồ đường đi giữa N địa điểm (có số hiệu là 1,2, ...N) trong thành phố X được cho bởi bảng vuông A kích thước N x N. Mỗi phần tử của bảng A chỉ có thể là số 0 hoặc số 1. Nếu phần tử ở hàng i cột j của bảng A mà bằng 1 thì có đường đi từ địa điểm i tới địa điểm j, ngược lại nếu không có đường đi ($1 \leq i, j \leq N$).

Hãy liệt kê tất cả các đường đi khác nhau giữa hai địa điểm P và Q trong thành phố X. Số hiệu của mỗi địa điểm xuất hiện trên mỗi đường đi qua không quá 1 lần.

Dữ liệu vào là tệp văn bản DUONG.INP có cấu trúc:

- Dòng đầu tiên ghi số N ($0 < N \leq 100$).
- N dòng tiếp theo, mỗi dòng ghi N số là các giá trị của bảng A.
- Dòng cuối cùng ghi 2 số P và Q.
- Các số ghi trên một dòng cách nhau ít nhất là một ký tự trống.

Dữ liệu ra là tệp văn bản DUONG.OUT ghi lại các đường đi có thể có giữa hai địa điểm P và Q. Mỗi đường đi ghi trên một dòng. Nếu không có đường đi giữa hai địa điểm P và Q thì ghi thông báo “Không có đường đi”.

Ví dụ:

Tệp DUONG.INP	Tệp DUONG.OUT
3	1 -> 2 -> 3
0 1 1	1 -> 3
1 0 1	
1 1 0	
1 3	

Bài toán. Bài toán rút tiền tự động ATM 2

Một máy ATM hiện có n ($n \leq 30$) tờ tiền có mệnh giá $t_1, t_2, t_3, \dots, t_n$. Hãy đưa ra cách trả tiền đúng bằng s , sao cho cần ít tờ tiền nhất.

Input: Tập ATM.INP có dạng:

- Dòng đầu là hai số n và s .
- Dòng thứ hai gồm n số $t_1, t_2, t_3, \dots, t_n$

Output: Tập ATM.OUT có dạng: Nếu có thể trả đúng s thì đưa ra cách trả, dòng đầu tiên ghi số tờ, dòng tiếp theo ghi mệnh giá các đồng tiền cần trả. Nếu không ghi -1.

ATM.INP	ATM.OUT
11 490	200 20 20 50 100 100
20 200 10 20 20 50 50 50 50 100 100	

Bài toán cái túi 2. (Xếp va Li, xếp Ba lô)

Có N loại vật (mặt hàng), số lượng của mỗi loại là không hạn chế, với mỗi vật ta biết trọng lượng và giá trị của nó. Hãy xác định các vật cần để xếp vào một ba lô có sức chứa tối đa là M sao cho giá trị chứa trong ba lô là lớn nhất.

Dữ liệu vào: Tập văn bản **balo.inp**:

- Dòng đầu tiên: hai giá trị nguyên dương N và M .
- N dòng tiếp theo, mỗi dòng chứa hai giá trị nguyên dương d và v cho mỗi vật, trong đó d là trọng lượng, v là giá trị tính theo một đơn vị trọng lượng của vật đó (đơn giá). Các số cách nhau qua dấu cách.

Dữ liệu ra: Tập văn bản tên **balo.out**:

- Dòng đầu tiên ghi giá trị đạt được
- Các dòng tiếp theo, mỗi dòng ghi số thứ tự vật cần lấy và số lần lấy

Ví dụ:

balo.inp	balo.out
5 14	32.00
3 5	4 4
4 2	3 1
2 4	
3 7	
5 1	

MỘT SỐ BÀI TẬP KHÁC

Bài 1. Tạo sơn tổng hợp (HSG 11– 2011)

Từ N loại sơn ban đầu có số hiệu là $1, 2, \dots, N$ ($1 \leq N \leq 9$), người ta có thể tạo ra rất nhiều loại sơn tổng hợp khác nhau, bằng cách trộn lẫn một số loại sơn nào đó lại với nhau theo một liều lượng nào đó của mỗi loại. Khi tham gia trộn để được một loại sơn tổng hợp nào đó, các loại sơn khác nhau được đưa vào từ các vị trí khác nhau và liều lượng của mỗi loại sơn là bao nhiêu phụ thuộc vào vị trí đưa vào của các loại sơn đó. Liều lượng của mỗi loại sơn mà khác nhau trong khi trộn thì cho ra các loại sơn tổng hợp khác nhau. Hãy liệt kê ra tất cả các phương án trộn M loại sơn ($M \leq N$) trong N loại sơn đã cho để có được các loại sơn tổng hợp.

Dữ liệu vào: Giá trị của hai số N, M được nhập từ bàn phím

Dữ liệu ra: Tập SON.out có cấu trúc:

- Mỗi dòng ghi số hiệu của M loại sơn theo thứ tự khi đưa ra trộn để tạo ra một loại sơn tổng hợp nào đó. Dòng cuối cùng ghi số lượng các loại sơn tổng hợp tạo ra.

Ví dụ:

	SON.OUT
N=3	1 2
M=2	1 3
	2 1
	2 3
	3 1
	3 2
	6

Bài 2: Vận chuyển (HSG 11, 2016-2017)

Công ti vận tải X chuyên nhận các đơn hàng vận chuyển hàng hóa đi khắp nơi theo yêu cầu của khách hàng. Đơn hàng của công ti lần này là vận chuyển hàng từ cảng Sơn Dương – Hà Tĩnh đi Viên Chăn – Lào. Đơn hàng gồm n mặt hàng, mỗi mặt hàng có trọng lượng là a_i . Để tăng lợi nhuận và đảm bảo an toàn tuyệt đối trong mỗi lần vận chuyển, lựa chọn các mặt hàng có tổng trọng lượng lớn nhất và nhỏ hơn giới hạn tải cho phép.

Yêu cầu: Chọn trong n mặt hàng một số mặt hàng sao cho tổng trọng lượng của chúng là lớn nhất và nhỏ hơn giới hạn tải s .

Dữ liệu: Vào từ file văn bản TRANSPORT.INP

- Dòng đầu tiên gồm 2 số nguyên dương n và s ;
- Dòng tiếp theo gồm n số nguyên dương a_i là trọng lượng của các mặt hàng.

Các số trên cùng một dòng, ghi cách nhau ít nhất là một dấu cách.

Kết quả: Ghi ra file văn bản TRANSPORT.OUT một số duy nhất là tổng trọng lượng lớn nhất của các mặt hàng được chọn.

Ví dụ:

TRANSPORT.INP	TRANSPORT.OUT
---------------	---------------

5 16 3 4 2 5 6	15
-------------------	----

Ràng buộc:

- 80% số test ứng với 80% số điểm của bài có $n \leq 30$;
- 20% số test ứng với 20% số điểm của bài có $n \leq 10^3$, $s \leq 10^5$.

Bài 3. Chọn quà

Nhân dịp kỷ niệm 75 năm ngày thành lập Đoàn TNCS Hồ Chí Minh, Ban chấp hành Đoàn trường THPT X tổ chức một trò chơi trên lưới ô vuông cho các đội chơi. Lưới ô vuông có kích thước $n \times n$. Các ô của lưới được đánh số từ 1 đến n từ trên xuống dưới, các cột của lưới được đánh số từ 1 đến n từ trái sang phải. Ở mỗi ô nằm trên giao của dòng i ($1 \leq i \leq n$) và cột j ($1 \leq j \leq n$) của lưới, trên mỗi ô (i, j) của lưới ghi một số nguyên dương a_{ij} ($1 \leq i, j \leq n$) chính là giá trị của quà tại ô (i, j) đó.

Nhiệm vụ của người chơi là xuất phát tại ô $(1, 1)$ bên trái của lưới, mỗi lần chỉ được di chuyển sang ô bên phải kế tiếp hoặc di chuyển xuống ô ngay phía dưới của mình (khi ô đó nằm trong lưới và giá trị tại ô đó lớn hơn hoặc bằng giá trị tại ô hiện tại).

Lấy các di chuyển mà từ ô bất kỳ các lưới được phép di chuyển sang ô bên phải có giá trị không nhỏ hơn giá trị ở ô đó.

Yêu cầu: Đếm xem có bao nhiêu cách di chuyển theo quy tắc trên.

Dữ liệu: Vào từ tệp văn bản **QUA.INP**

Dòng đầu tiên chứa số nguyên dương n .

Dòng thứ i trong số n dòng tiếp theo chứa các số nguyên $a_{i1}, a_{i2}, \dots, a_{in}$ ($1 \leq i \leq n$).

Các số trên cùng một dòng được ghi cách nhau ít nhất một dấu cách.

Kết quả: Ghi ra tệp văn bản **QUA.OUT** một số duy nhất là số cách di chuyển theo quy tắc trên. Nếu không có cách nào di chuyển thì ghi 0.

Ví dụ:

QUA.INP	QUA.OUT
3 1 2 3 2 4 1 3 3 2	3

Ràng buộc:

Có 60% số test ứng với 60% số điểm của bài có: $1 \leq n \leq 10$, $a_{ij} \leq 30$.

Có 40% số test còn lại ứng với 40% số điểm của bài có: $n \leq 20$, $a_{ij} \leq 10^3$.

Bài 4: CHỌN ĐỒ VẬT (HSG 12, 2018-2019)

Knapsack là một bài toán nổi tiếng về độ khó: Hiện chưa có một lời giải hiệu quả cho nghiệm tối ưu trong trường hợp tổng quát. Những cố gắng để giải quyết bài toán Knapsack đã cho ra đời nhiều thuật toán gần đúng, hoặc những thuật toán tối ưu trong trường hợp đặt biệt.

Bài toán được phát biểu như sau: Cho n đồ vật, đồ vật thứ i có trọng lượng là w_i và giá trị là v_i ($w_i, v_i \in \mathbb{Z}^+$). Cho một balô có giới hạn trọng lượng là m , hãy chọn ra một số đồ vật cho vào balô sao cho tổng trọng lượng của chúng không vượt quá m và tổng giá trị của chúng là lớn nhất có thể.

Dữ liệu: Vào từ file văn bản KSACK.INP

- Dòng 1 chứa hai số nguyên dương n, m ;
- n dòng tiếp theo, dòng thứ i chứa hai số nguyên dương w_i, v_i cách nhau ít nhất một dấu cách ($w_i, v_i \leq 1000$)

Kết quả: Ghi ra file văn bản KSACK.OUT

- Dòng 1: Ghi tổng giá trị lớn nhất có thể chọn được.
- Dòng 2: Ghi chỉ số những đồ vật được chọn, nếu có nhiều phương án đúng thì chỉ ghi ra một cách chọn.

Ví dụ:

KSACK.INP	KSACK.OUT
5 11	110
3 30	1 2 5
4 40	
5 40	
9 100	
4 40	

Ràng buộc:

- Có 60% số test khác ứng với 60% số điểm của bài có $n \leq 20$, $m \leq 10^9$
- Có 20% số test khác ứng với 20% số điểm của bài có $20 < n \leq 40$, $m \leq 10^9$
- Có 20% số test còn lại với 20% số điểm của bài có $40 < n \leq 10^3, m \leq 10^3$

Bài 5: Biểu thức bằng K (HSG 11, 2017-2018)

Cho dãy số nguyên dương $a_1, a_2, \dots, a_n$, người ta viết các phần tử của dãy số này cách nhau bởi dấu $?$: $a_1?a_2?...?a_n$

Hãy thay các dấu $?$ bởi dấu $+$ hoặc $-$ để được một biểu thức có giá trị bằng k cho trước. Biết rằng luôn tồn tại cách làm như vậy.

Dữ liệu vào từ tệp văn bản `EXPR.INP` có cấu trúc:

- Dòng 1 chứa số nguyên dương n ($2 \leq n \leq 100$) và số nguyên k ;
- Dòng 2 chứa n số nguyên dương $a_1, a_2, \dots, a_n$ (Ví: $a_i \leq 100$);
- Các số trên một dòng được ghi cách nhau ít nhất một dấu cách.

Kết quả ghi vào tệp văn bản `EXPR.OUT` một dòng gồm $n - 1$ ký tự, ký tự thứ $i \in \{+, -\}$ là dấu điền vào vị trí dấu $?$ thứ i .

Ví dụ:

EXPR.INP	EXPR.OUT
4 2 1 2 3 4	++-

Ràng buộc: Có 60% số test ứng với 60% số điểm của bài có $n \leq 20$.

Cuộc đua (HSG 11, 2018-2019)

Nội dung cuối cùng trong triển lãm là robot vượt chướng ngại vật. Trước khi cán đích cuộc đua những con robot là các con robot phải vượt qua một vùng chướng ngại vật hình chữ nhật kích thước $n \times m$ ô (n hàng và m cột). Các con robot có thể vào một ô bất kỳ ở hàng đầu tiên và không được ra khỏi vùng chướng ngại vật trước khi tới được hàng cuối cùng. Từ một ô (i, j) , robot có thể di chuyển sang một trong số các ô $(i+1, j-1)$, $(i+1, j)$, $(i+1, j+1)$ nếu ô đó tồn tại. Thời gian vượt qua ô (i, j) là $a_{i,j}$, $i = 1, 2, \dots, n$, $j = 1, 2, \dots, m$.

Yêu cầu: Hãy xác định thời gian nhỏ nhất một con robot có thể vượt qua vùng chướng ngại vật.

Dữ liệu: Vào từ file văn bản RACE.INP:

- Dòng đầu tiên chứa 2 số nguyên n và m ($1 \leq n, m \leq 2000$);
- Dòng thứ i trong n dòng tiếp theo chứa m số nguyên $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($0 \leq a_{i,j} \leq 10^9$, $j = 1, 2, \dots, m$).

Kết quả: Ghi ra file văn bản RACE.OUT gồm:

- Dòng đầu tiên ghi một số nguyên là thời gian nhỏ nhất tìm được;
- Dòng tiếp theo ghi n số nguyên là giá trị của các ô (i, j) mà robot đã đi qua theo thứ tự từ hàng 1 đến hàng n .

Các số trên cùng dòng ghi cách nhau ít nhất một dấu cách.

Chú ý:

- Bài toán có thể có nhiều đường đi thì chỉ cần ghi ra giá trị của một đường đi bất kỳ.
- Nếu kết quả chỉ ghi được thời gian nhỏ nhất mà không ghi được đường đi thì đạt được 70% số điểm của test.

Ví dụ:

RACE.INP	RACE.OUT
3 4	11 5
6 5 1 3	2 4
2 8 7 8	
9 4 6 4	

Ràng buộc:

- Có 70% số test ứng với 70% số điểm của bài thỏa mãn điều kiện: $1 \leq n, m \leq 20$;
- 30% số test còn lại ứng với 30% số điểm của bài không có ràng buộc gì thêm.

Chọn quà (HSG 11, 2015-2016)

Nhân dịp kỷ niệm 75 năm ngày thành lập Đoàn TNCS Hồ Chí Minh, Ban chấp hành Đoàn trường THPT X tổ chức một trò chơi trên lưới ô vuông cho các đội chơi. Lưới ô vuông có kích thước $n \times n$. Các dòng của lưới được đánh số từ 1 đến n từ trên xuống dưới, các cột của lưới được đánh số từ 1 đến n từ trái qua phải. Ô nằm trên giao của dòng i , cột j được gọi là ô (i, j) của lưới. Trên mỗi ô (i, j) của lưới ghi một số nguyên dương a_{ij} , ($1 \leq i, j \leq n$) chính là giá trị của món quà đặt trên đó. Nhiệm vụ của người chơi là xuất phát từ ô $(1, 1)$ bên trái của lưới tìm cách di chuyển sang ô bên phải của lưới để lấy được nhiều món quà nhất về cho đội của mình (khi đi qua ô nào thì nhận được quà trên ô đó). Quy tắc di chuyển là từ một ô bất kỳ của lưới được phép di chuyển sang ô bên phải có giá trị không nhỏ hơn giá trị ô đó.

Yêu cầu: Đếm xem có bao nhiêu cách di chuyển theo quy tắc trên.

Dữ liệu: Vào từ file văn bản QUA.INP

- Dòng đầu tiên chứa số nguyên dương n .
- Dòng thứ i trong số n dòng tiếp theo chứa các số nguyên $a_{i1}, a_{i2}, \dots, a_{in}$ ($1 \leq i \leq n$). Các số trên cùng một dòng được ghi cách nhau ít nhất một dấu cách.

Kết quả: Ghi ra file văn bản QUA.OUT một số duy nhất là số cách di chuyển theo quy tắc trên. Nếu không có cách nào di chuyển thì ghi số 0.

Ví dụ:

QUA.INP	QUA.OUT
3 1 2 3 2 4 1 3 3 2	3

Ràng buộc:

- Có 60% số test ứng với 60% số điểm của bài có: $1 \leq n \leq 10$, $a_{ij} \leq 30$;
- Có 40% số test còn lại ứng với 40% số điểm của bài có: $n \leq 20$, $a_{ij} \leq 10^3$.